

《数学建模与实验》第8次作业

姓名：张致远

学号：2023213980

概率统计建模

题目 1：转移概率

吸收链的转移概率矩阵如上，请求出平均转移次数以及落到每个吸收状态的概率，并利用代码来实现。

Figure 1: 转移概率矩阵

1	0	0	0	0	0
0	1	0	0	0	0
1/4	0	1/2	0	1/4	0
0	0	0	0	1	0
1/16	1/16	1/4	1/8	1/4	1/4
0	1/4	0	0	1/4	1/2

答：

平均转移次数（从每个瞬时状态出发，进入吸收状态前的期望步数）：从状态 2 开始：4 步从状态 4 开始：4 步从状态 5 开始：4 步

落入各吸收状态的概率矩阵：

这是一个 3×3 矩阵，对应：

行表示：从瞬时状态 2、4、5 出发列表示：最终吸收到状态 0、1、3 的概率

$$B = \begin{bmatrix} 0.6875 & 0.1875 & 0.125 \\ 0.375 & 0.375 & 0.25 \\ 0.1875 & 0.6875 & 0.125 \end{bmatrix}$$

```
import numpy as np
```

```
# 定义转移概率矩阵
```

```
P = np.array([
    [1, 0, 0, 0, 0, 0],
    [0, 1, 0, 0, 0, 0],
    [1/4, 0, 1/2, 0, 1/4, 0],
    [0, 0, 0, 1, 0, 0],
    [1/16, 1/16, 1/4, 1/8, 1/4, 1/4],
    [0, 1/4, 0, 0, 1/4, 1/2]
])
```

```

        [1/16, 1/16, 1/4, 1/8, 1/4, 1/4],
        [0, 1/4, 0, 0, 1/4, 1/2]
    ])

# 提取瞬时状态和吸收状态的子矩阵
Q = P[:4, :4]
R = P[:4, 4:]

# 计算基本矩阵  $N$ 
N = np.eye(4) - Q

# 计算平均转移次数
mu = np.linalg.inv(np.eye(4) - N)

# 计算落到每个吸收状态的概率
pi = np.linalg.inv(np.eye(4) - Q) @ R

# 输出结果
print(" 平均转移次数矩阵 (mu):")
print(mu)
print("\n落到每个吸收状态的概率 (pi):")
print(pi)

```

题目 2：猪

某猪场 25 头育肥猪 4 个胴体性状的数据资料如下表，试进行 $\bar{y}$ 肉量 y 对眼肌面积 (x_1)、腿肉量 (x_2)、腰肉量 (x_3) 的多元统计回归分析。

答：

```

import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

# 数据准备
data = {
    'y': [15.02, 12.62, 14.86, 13.98, 15.91, 12.47, 15.80, 14.32, 13.76, 15.18, 14.20,
    'x1': [23.73, 22.34, 28.84, 27.67, 20.83, 22.27, 27.57, 28.01, 24.79, 28.96, 25.7,
    'x2': [5.49, 4.32, 5.04, 4.72, 5.35, 4.27, 5.25, 4.62, 4.42, 5.30, 4.87, 5.80, 5.

```

序号	瘦肉量 y (kg)	眼肌面积 x_1 (cm ²)	腿肉量 x_2 (kg)	腰肉量 x_3 (kg)	序号	瘦肉量 y (kg)	眼肌面积 x_1 (cm ²)	腿肉量 x_2 (kg)	腰肉量 x_3 (kg)
1	15.02	23.73	5.49	1.21	14	15.94	23.52	5.18	1.98
2	12.62	22.34	4.32	1.35	15	14.33	21.86	4.86	1.59
3	14.86	28.84	5.04	1.92	16	15.11	28.95	5.18	1.37
4	13.98	27.67	4.72	1.49	17	13.81	24.53	4.88	1.39
5	15.91	20.83	5.35	1.56	18	15.58	27.65	5.02	1.66
6	12.47	22.27	4.27	1.50	19	15.85	27.29	5.55	1.70
7	15.80	27.57	5.25	1.85	20	15.28	29.07	5.26	1.82
8	14.32	28.01	4.62	1.51	21	16.40	32.47	5.18	1.75
9	13.76	24.79	4.42	1.46	22	15.02	29.65	5.08	1.70
10	15.18	28.96	5.30	1.66	23	15.73	22.11	4.90	1.81
11	14.20	25.77	4.87	1.64	24	14.75	22.43	4.65	1.82
12	17.07	23.17	5.80	1.90	25	14.35	20.04	5.08	1.53
13	15.40	28.57	5.22	1.66					

要求

(1) 求 y 关于 x_1, x_2, x_3 的线性回归方程

$$y = c_0 + c_1 x_1 + c_2 x_2 + c_3 x_3,$$

计算 c_0, c_1, c_2, c_3 的估计值;

(2) 对上述回归模型和回归系数进行检验 (利用统计量说明回归的显著性, 并给出残差图);

(3) 试建立 y 关于 x_1, x_2, x_3 的二项式回归模型, 并根据适当统计量指标选择一个较好的模型。

```

        'x3': [1.21, 1.35, 1.92, 1.49, 1.56, 1.50, 1.85, 1.51, 1.46, 1.66, 1.64, 1.90, 1.0]
    }

df = pd.DataFrame(data)

# 定义自变量和因变量
X = df[['x1', 'x2', 'x3']]
y = df['y']

# 添加常数项
X = sm.add_constant(X)

# 拟合多元线性回归模型
model = sm.OLS(y, X).fit()

# 输出模型摘要
print(model.summary())

# 绘制残差图
residuals = model.resid
plt.scatter(model.fittedvalues, residuals)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted')
plt.show()

# 建立二项式回归模型
X_poly = np.column_stack((X['x1'], X['x2'], X['x3'], X['x1']**2, X['x2']**2, X['x3']**2))
model_poly = sm.OLS(y, sm.add_constant(X_poly)).fit()

# 输出多项式模型摘要
print(model_poly.summary())

# 比较模型
print("Linear Model R2:", model.rsquared)
print("Polynomial Model R2:", model_poly.rsquared)

=====

```

```

Dep. Variable:          y    R-squared:
                        0.844
Model:                  OLS    Adj. R-squared:
                        0.821
Method:                 Least Squares    F-statistic:
                        37.75
Date:                   Sat, 17 May 2025    Prob (F-statistic):      1.22e-08
Time:                   16:52:02    Log-Likelihood:
                        -13.871
No. Observations:       25    AIC:
                        35.74
Df Residuals:           21    BIC:
                        40.62
Df Model:               3

Covariance Type:        nonrobust

=====
                        coef    std err          t      P>|t|      [0.025    0.975]
-----
const                0.8539      1.372      0.622      0.540      -2.000      3.707
x1                   0.0178      0.029      0.609      0.549      -0.043      0.078
x2                   2.0782      0.268      7.741      0.000       1.520      2.637
x3                   1.9396      0.510      3.806      0.001       0.880      2.999
=====

Omnibus:              1.760    Durbin-Watson:
                        2.407
Prob(Omnibus):         0.415    Jarque-Bera (JB):      1.296
Skew:                  0.337    Prob(JB):
                        0.523
Kurtosis:              2.110    Cond. No.
                        400.

=====

```

题目 3：蠓虫分类判别已知 6 个 Apf 蠓虫与 9 个 Af 蠓虫的触角长和翅长，根据样本数据建立数学模型，正确区分两类蠓虫；用建立的模型识别已知触角长和翅长的 3 个待判蠓虫样本；如果 Apf 蠓虫是某种疾病的载体毒蠓，Af 蠓虫是传粉益蠓，如何修改分类判别法再进行识别。

Apf蠓虫样本		Af蠓虫样本		待判样本 x	
触角长	翅长	触角长	翅长	触角长	翅长
1.14	1.78	1.24	1.72	1.24	1.80
1.18	1.96	1.36	1.74	1.29	1.81
1.20	1.86	1.38	1.64	1.43	2.03
1.26	2.00	1.38	1.82		
1.28	2.00	1.38	1.90		
1.30	1.96	1.40	1.70		
		1.48	1.82		
		1.54	1.82		
		1.56	2.08		

答：

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import train_test_split

# 整理数据
X = np.array([[1.14, 1.78], [1.18, 1.96], [1.20, 1.86], [1.26, 2.00], [1.28, 2.00], [
    1.24, 1.72], [1.36, 1.74], [1.38, 1.64], [1.38, 1.82], [1.40, 1.90], [
y = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]) # Af 为 1, Apf 为 0

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=

# 创建 LDA 模型
lda = LDA()

# 拟合模型
lda.fit(X_train, y_train)

# 预测
predictions = lda.predict(X_test)

# 预测待判样本
samples = np.array([[1.24, 1.80], [1.29, 1.81], [1.43, 2.03]])
sample_predictions = lda.predict(samples)
print(" 预测类别: ", sample_predictions)
```

预测类别: [0 1 1]